

Tools for Data Science: Python

4. เครื่องมือวิทยาการข้อมูล: Python #2

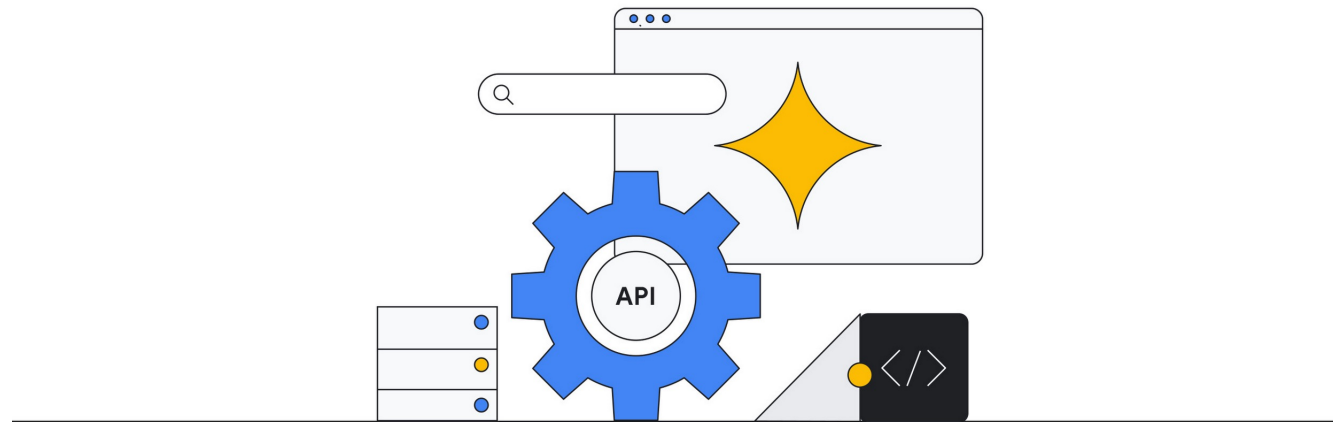
4.1 REST API

4.2 Webscraping

4.3 Introducing Scikit-Learn



Application Programming Interface: API



API Library

```
import pandas as pd
```

```
df.head()
```

	a	b
0	11	12
1	21	22
2	31	32

```
df.sum()
```

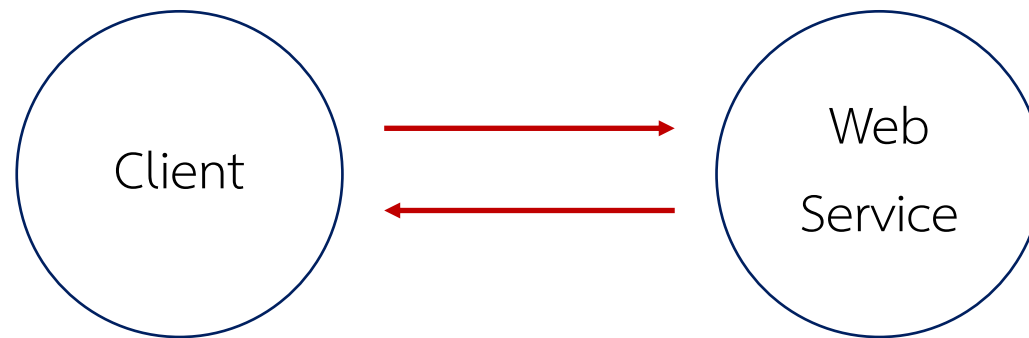
โปรแกรมที่
เขียน

`df.head()`

Pandas

REST API Library

- ตัวอย่าง CoinGecko API
- ใช้ PyCoinGecko



```
#pip install pycoingecko
```

```
from pycoingecko import CoinGeckoAPI
cg = CoinGeckoAPI()
btc_data = cg.get_coin_market_chart_by_id(id='bitcoin',
                                           vs_currency='usd', days=10)

btc_data
```

```
{'prices': [[1687784433348, 30305.29805574046],
            [1687788018053, 30448.975759126337],
            [1687791628389, 30393.01984154619],
            [1687795212684, 30443.052698610674],
            [1687798832718, 30196.355307137776],
            [1687802466026, 30108.572943514617],
            [1687806021484, 30211.898951847645],
            [1687809617100, 30259.427747406215],
            [1687813243987, 30170.700475086625],
            [1687816822159, 30160.11413803748],
            [1687820413335, 30238.170753896567],
```

```
btc_data = btc_data['prices']  
data = pd.DataFrame(btc_data, columns=['TimeStamp', 'Price'])
```

	TimeStamp	Price
0	1687784433348	30305.298056
1	1687788018053	30448.975759
2	1687791628389	30393.019842
3	1687795212684	30443.052699
4	1687798832718	30196.355307
...	...	...
236	1688634041817	31139.900723
237	1688637627365	31128.580360
238	1688641283256	31144.528276
239	1688644811511	31105.539283
240	1688645151000	30785.813005
241 rows x 2 columns		

```
data['Date'] = pd.to_datetime(data['TimeStamp'], unit='ms')
data
```

	TimeStamp	Price	Date
0	1687784433348	30305.298056	2023-06-26 13:00:33.348
1	1687788018053	30448.975759	2023-06-26 14:00:18.053
2	1687791628389	30393.019842	2023-06-26 15:00:28.389
3	1687795212684	30443.052699	2023-06-26 16:00:12.684
4	1687798832718	30196.355307	2023-06-26 17:00:32.718
...	...	...	...
236	1688634041817	31139.900723	2023-07-06 09:00:41.817
237	1688637627365	31128.580360	2023-07-06 10:00:27.365
238	1688641283256	31144.528276	2023-07-06 11:01:23.256
239	1688644811511	31105.539283	2023-07-06 12:00:11.511
240	1688645151000	30785.813005	2023-07-06 12:05:51.000
241 rows x 3 columns			

REST API HTML request

```
import requests
api_url = "url_rest"
response = requests.get(api_url)
response.json()
```

- ทดสอบเรียก REST API ด้วยโปรแกรม POSTMAN จากตัวอย่าง URL
“https://api.coingecko.com/api/v3/coins/markets?vs_currency=usd&order=market_cap_desc&per_page=10&page=1&sparkline=false&price_change_percentage=7d”

```
[{'id': 'bitcoin',
  'symbol': 'btc',
  'name': 'Bitcoin',
  'image': 'https://assets.coingecko.com/coins/images/1/large/bitcoin.png?1547033579',
  'current_price': 30590,
  'market_cap': 594592425956,
  'market_cap_rank': 1,
  'fully_diluted_valuation': 642920118996,
  'total_volume': 12670027972,
  'high_24h': 31480,
  'low_24h': 30236,
  'price_change_24h': 207.42,
  'price_change_percentage_24h': 0.68269,
  'market_cap_change_24h': 4318696314,
```

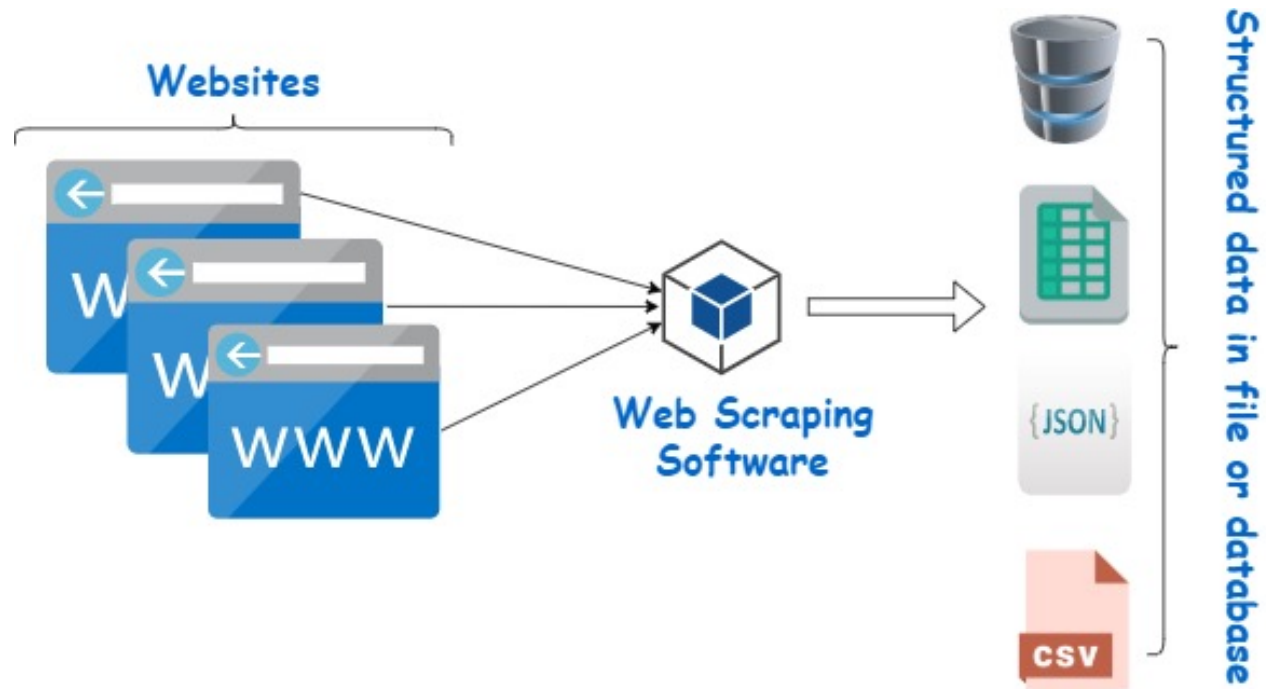
การทดลอง REST API

- เปรียบเทียบราคา Crypto บน coingecko โดยจัดอันดับ market_cap_rank = 1-5
- ใช้ API Library: CoinGecko API เปรียบเทียบกับวิธี HTML request
- นำเสนอในรูปแบบ Pandas Dataframe และ Plot market_cap เทียบ 5 อันดับ



Web Scrapping

- หากข้อมูล html ที่ต้องการ ไม่สามารถเรียกใช้ด้วย API



<https://www.webharvy.com/articles/what-is-web-scraping.html>

ตัวอย่าง BeautifulSoup Web Scraping

```
%%html
<!DOCTYPE html>
<html>
<head>
<title>Page Title</title>
</head>
<body>
<h3><b id='boldest'>Lebron James</b></h3>
<p> Salary: $ 92,000,000 </p>
<h3> Stephen Curry</h3>
<p> Salary: $85,000, 000 </p>
<h3> Kevin Durant </h3>
<p> Salary: $73,200, 000</p>
</body>
</html>
```

✓ 0.0s

Lebron James

Salary: \$ 92,000,000

Stephen Curry

Salary: \$85,000, 000

Kevin Durant

Salary: \$73,200, 000

```
html="<!DOCTYPE
html><html><head><title>Page
Title</title></head><body><h3><b
id='boldest'>Lebron James</b></h3><p>
Salary: $ 92,000,000 </p><h3> Stephen
Curry</h3><p> Salary: $85,000, 000
</p><h3> Kevin Durant </h3><p> Salary:
$73,200, 000</p></body></html>"
```

```
soup = BeautifulSoup(html, 'html5lib')
print(soup.prettify())
```

```
<!DOCTYPE html>
<html>
  <head>
    <title> Page Title </title>
  </head>
```

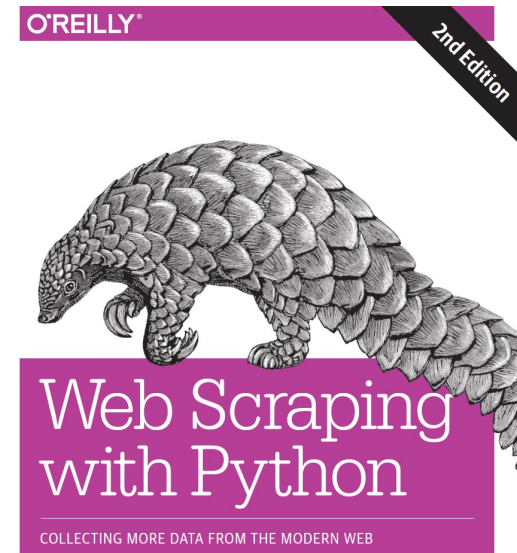
...

Beautiful Soup Web Scraping

```
from urllib.request import urlopen
from bs4 import BeautifulSoup
html = urlopen('http://www.pythonscraping.com/pages/page1.html')
bs = BeautifulSoup(html.read(), 'html.parser') ## Try html5lib
print(bs.h1)
```

```
<h1>An Interesting Title</h1>
```

```
Try
bs.html.body.h1
bs.body.h1
bs.html.h1
```



Ryan Mitchell

- ตรวจสอบ
ข้อผิดพลาด

```
from urllib.request import urlopen
from urllib.error import HTTPError
from bs4 import BeautifulSoup
def getTitle(url):
    try:
        html = urlopen(url)
    except HTTPError as e:
        return None
    try:
        bs = BeautifulSoup(html.read(), 'html.parser')
        title = bs.body.h1
    except AttributeError as e:
        return None
    return title

title = getTitle('http://www.pythonscraping.com/pages/page1.html')
if title == None:
    print('Title could not be found')
else:
    print(title)
```

Try URL:

<https://www.pythonscraping.com/pages/warandpeace.html>

การทดลอง Web Scraping

Introducing Scikit-Learn



Introducing Scikit-Learn

- เตรียมอัลกอริทึมการเรียนรู้ของเครื่องไว้หลากหลาย
 - Supervised learning
 - Unsupervised learning
- การเตรียมข้อมูลก่อนเข้าแบบจำลอง
- ตัวเลือก Hyperparameters ของแบบจำลอง
- สร้าง Data Pipeline
- <https://scikit-learn.org/>

Introducing Scikit-Learn: Linear Regression

```
In [5]: import matplotlib.pyplot as plt
import numpy as np
rng = np.random.RandomState(42)
x = 10 * rng.rand(50)
y = 2 * x - 1 + rng.randn(50)
plt.scatter(x, y);
```

```
In [6]: from sklearn.linear_model import LinearRegression
```

```
In [7]: model = LinearRegression(fit_intercept=True)
model
```

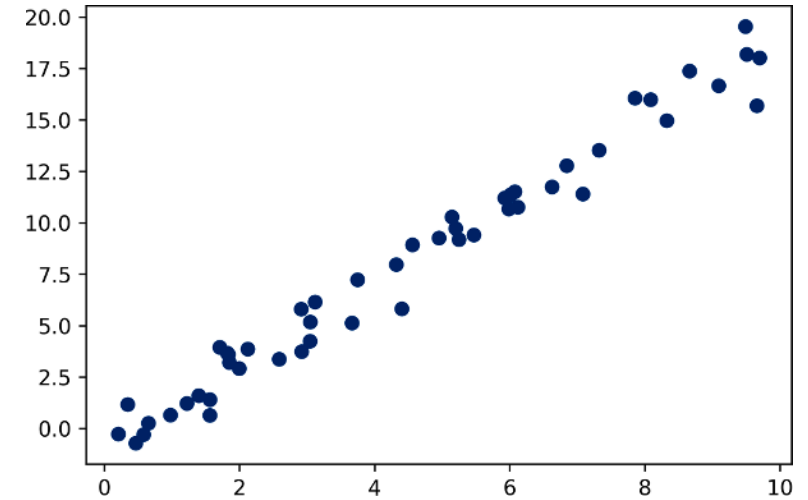
```
Out[7]: LinearRegression()
```

```
In [8]: X = x[:, np.newaxis]
X.shape
```

```
Out[8]: (50, 1)
```

```
In [9]: model.fit(X, y)
```

```
Out[9]: LinearRegression()
```



Introducing Scikit-Learn: Linear Regression

```
In [10]: model.coef_  
Out[10]: array([1.9776566])
```

$$y = a_0 + a_1x_1$$

```
In [11]: model.intercept_  
Out[11]: -0.9033107255311146
```

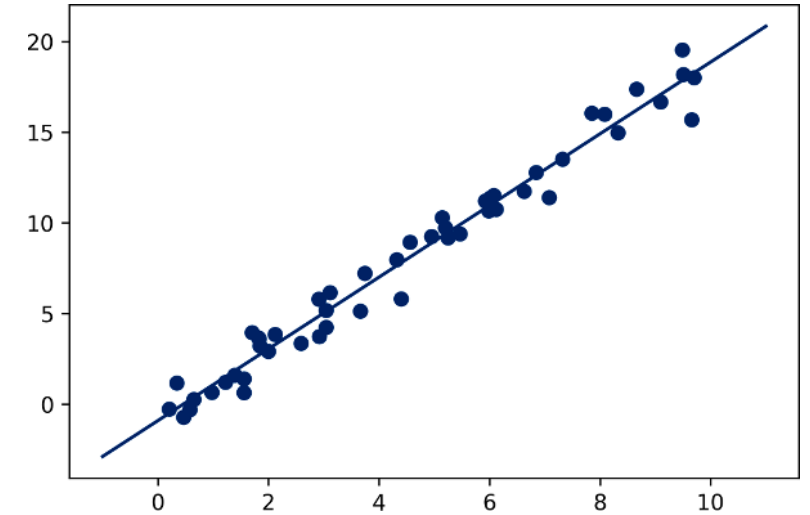
```
In [12]: xfit = np.linspace(-1, 11)  
In [13]: Xfit = xfit[:, np.newaxis]  
         yfit = model.predict(Xfit)
```

```
In [14]: plt.scatter(x, y)  
         plt.plot(xfit, yfit);
```

```
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
```

```
mse = mean_squared_error(y, yfit)  
print(mse)
```

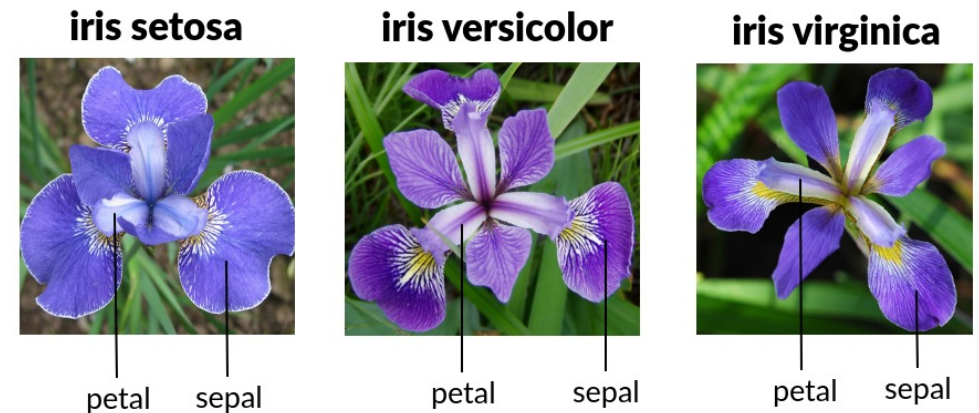
```
87.0410288904571
```



$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

Introducing Scikit-Learn: Gaussian Naïve Bayes Classification

- แบบจำลองอย่างง่ายใช้ในการจำแนก สร้างจาก Bayes Theorem
- ข้อมูล iris



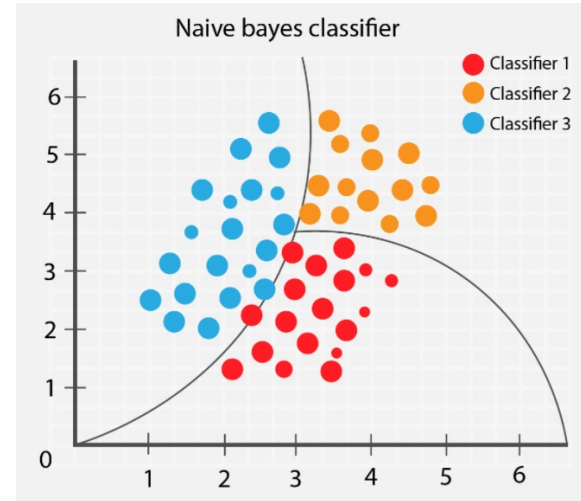
```
In [1]: import seaborn as sns  
iris = sns.load_dataset('iris')  
iris.head()
```

```
Out[1]:
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

ทฤษฎีเบย์ (Bayes' Theorem)

- ความน่าจะเป็นในการเกิดสิ่งหนึ่ง ก็ต่อเมื่ออีกสิ่งที่ได้เกิดขึ้น
- $P(A|B)$ = ความน่าจะเป็นของเหตุการณ์ A เมื่อเกิดเหตุการณ์ B แล้ว



<https://kdagiit.medium.com/naive-bayes-algorithm-4b8b990c7319>

- c คือ Class
- x คือ Attribute
- P คือ Probability (ความน่าจะเป็น)

$$P(c|x) = \frac{P(x|c)P(c)}{P(x)}$$

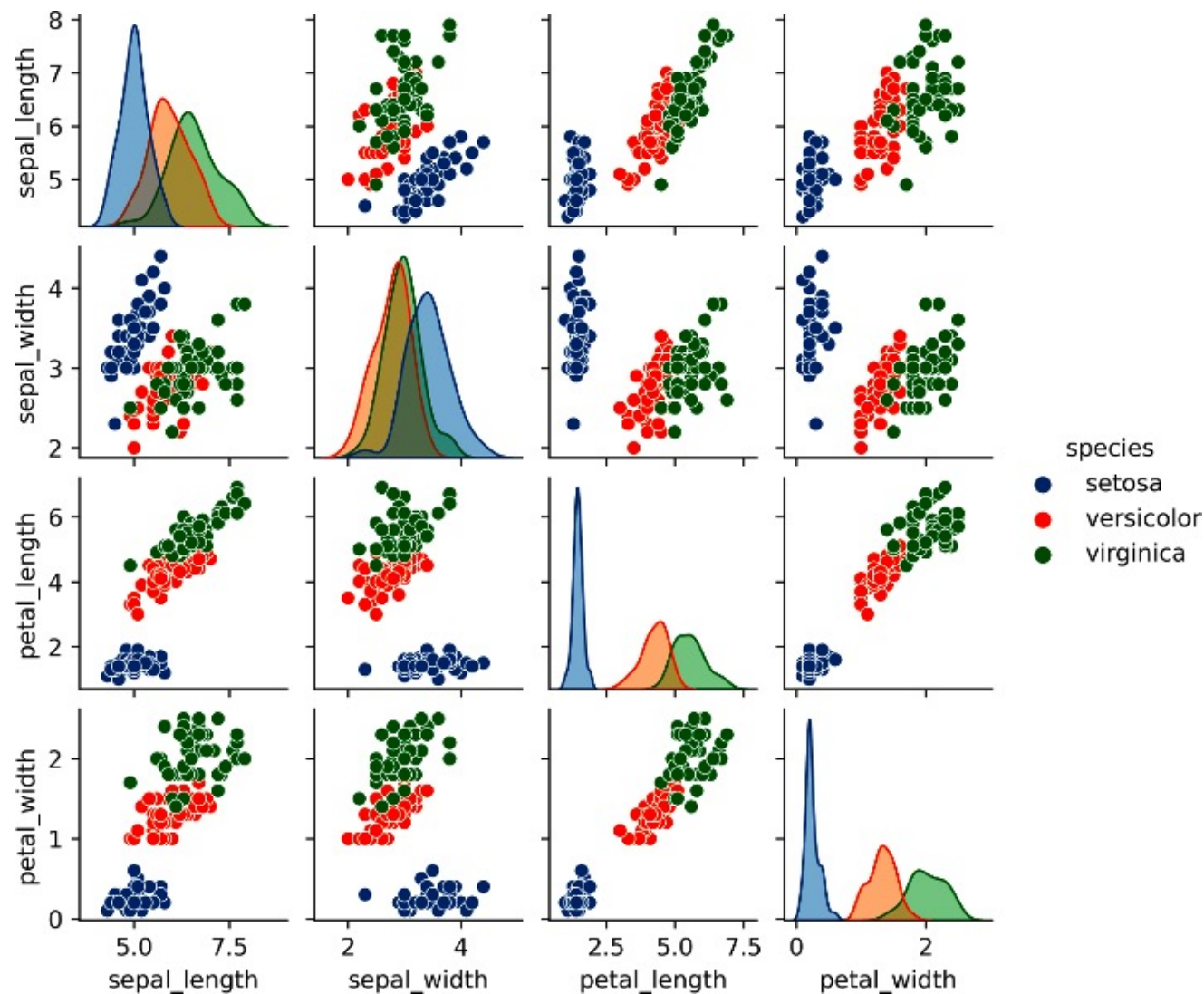
Diagram labels for the equation above:

- $P(c|x)$ is labeled **Posterior Probability** (with a downward arrow).
- $P(x|c)$ is labeled **Likelihood** (with an upward arrow).
- $P(c)$ is labeled **Class Prior Probability** (with an upward arrow).
- $P(x)$ is labeled **Predictor Prior Probability** (with a downward arrow).

$$P(c|X) = P(x_1|c) \times P(x_2|c) \times \dots \times P(x_n|c) \times P(c)$$

Introducing Scikit-Learn: Gaussian Naïve Bayes Classification

```
In [2]: %matplotlib inline
import seaborn as sns
sns.pairplot(iris,
hue='species',
height=1.5);
```



Introducing Scikit-Learn:

Gaussian Naïve Bayes

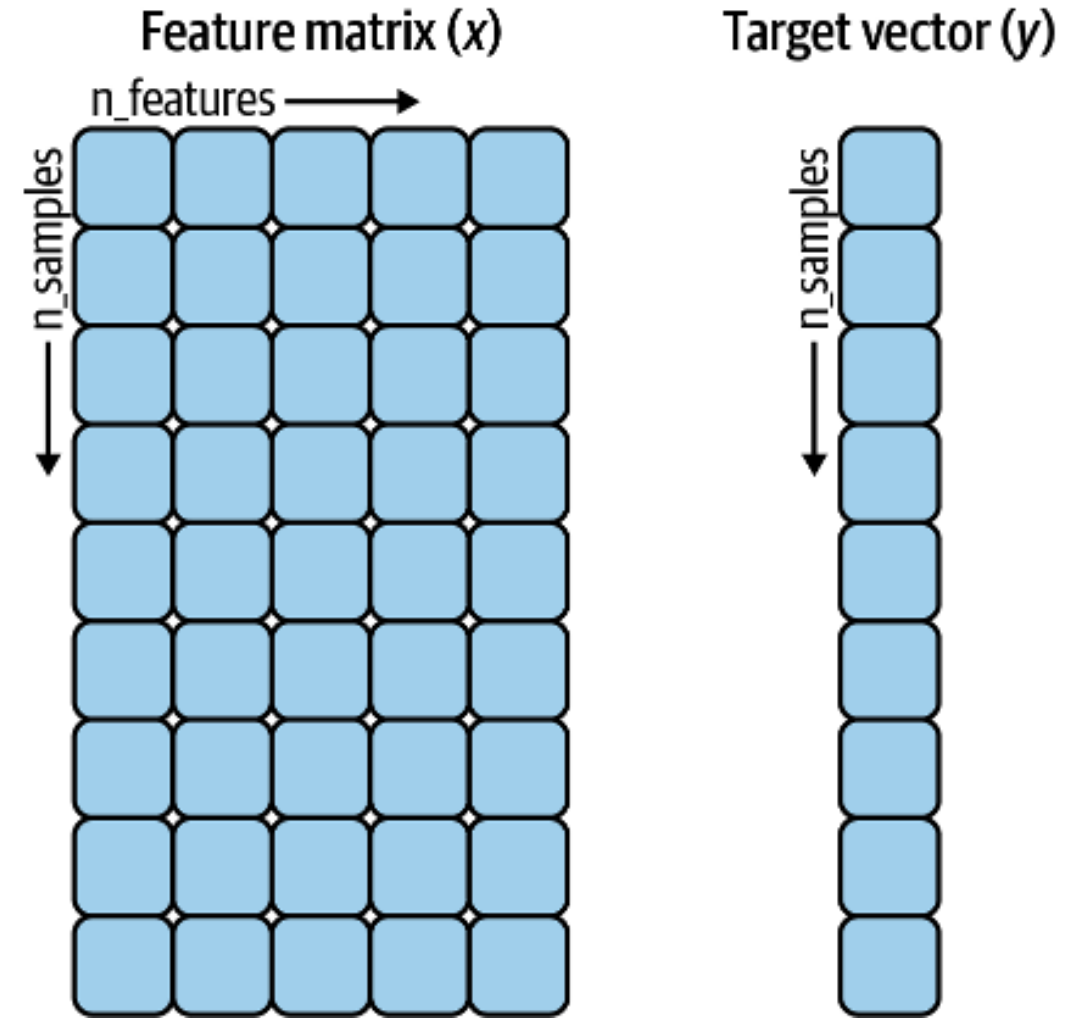
Classification

```
In [3]: X_iris = iris.drop('species',  
                             axis=1)
```

```
Out[3]: X_iris.shape  
(150, 4)
```

```
In [4]: y_iris = iris['species']  
y_iris.shape
```

```
Out[4]: (150,)
```



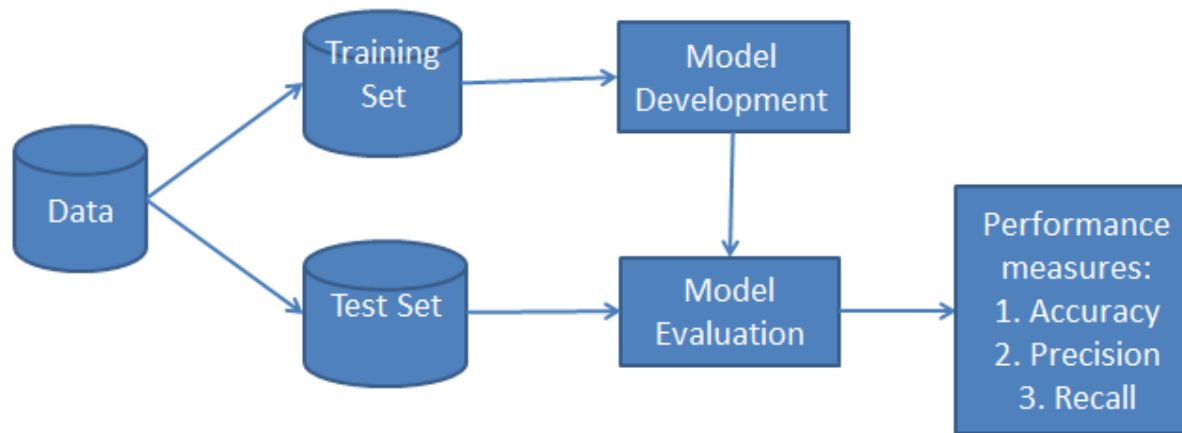
Introducing Scikit-Learn: Gaussian Naïve Bayes Classification

```
In [15]: from sklearn.model_selection import train_test_split
Xtrain, Xtest, ytrain, ytest = train_test_split(X_iris, y_iris,
                                              random_state=1)
```

```
In [16]: from sklearn.naive_bayes import GaussianNB # 1. choose model class
model = GaussianNB() # 2. instantiate model
model.fit(Xtrain, ytrain) # 3. fit model to data
y_model = model.predict(Xtest) # 4. predict on new data
```

```
In [17]: from sklearn.metrics import accuracy_score
accuracy_score(ytest, y_model)
```

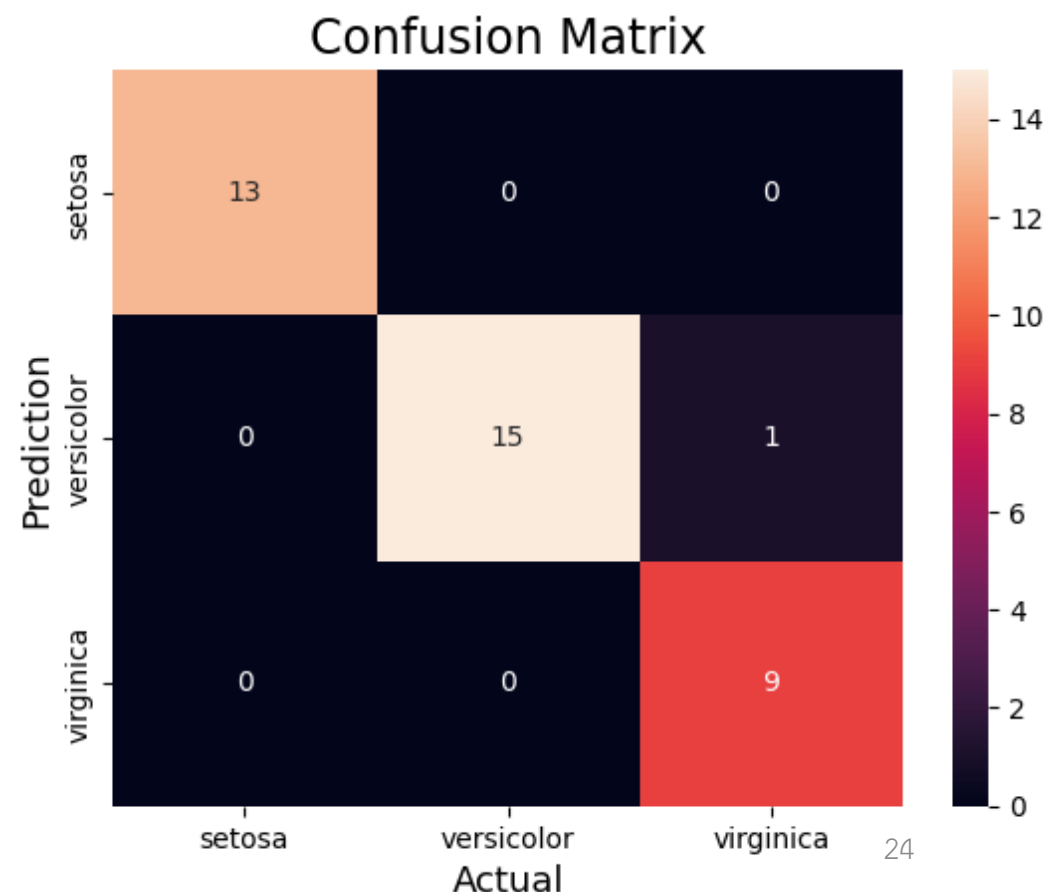
```
Out[17]: 0.9736842105263158
```



Introducing Scikit-Learn: Gaussian Naïve Bayes Classification

```
from sklearn.metrics import confusion_matrix
# Plotting a Confusion Matrix in Seaborn
conf_matrix = confusion_matrix(ytest, y_model, labels=model.classes_)
sns.heatmap(conf_matrix,
             annot=True,
             fmt='g',
             xticklabels=model.classes_,
             yticklabels=model.classes_,
             )

plt.ylabel('Prediction', fontsize=13)
plt.xlabel('Actual', fontsize=13)
plt.title('Confusion Matrix', fontsize=17)
plt.show()
```



Model Validation

- Re-substitution
- Hold-out
- K-folds Cross-Validation

ตัวอย่างแบบจำลอง K-nearest neighbor

- เพื่อนบ้านที่ใกล้เคียงที่สุด K ตัว
- ดึงข้อมูลที่ใกล้เคียงตัวเองที่สุด K ตัว แล้วนับจำนวนเพื่อนบ้านที่ใกล้เคียงทั้งหมดใน K ตัว เป็น Class อะไรมากที่สุด

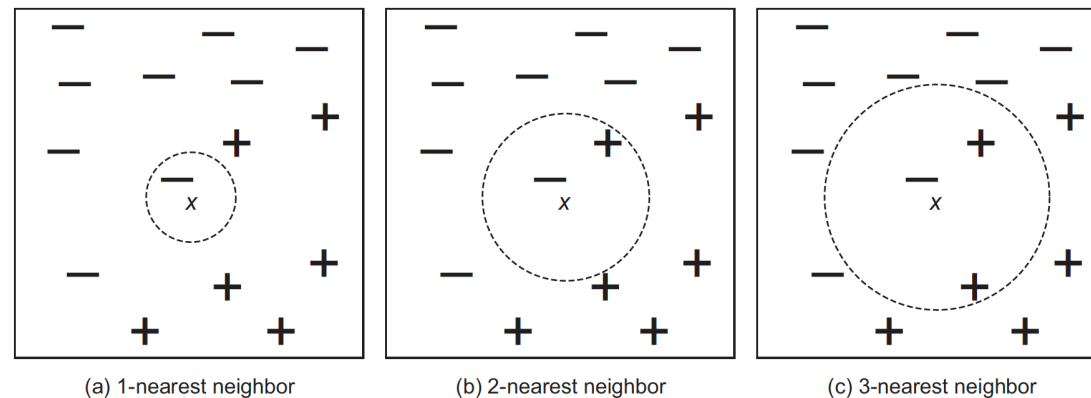
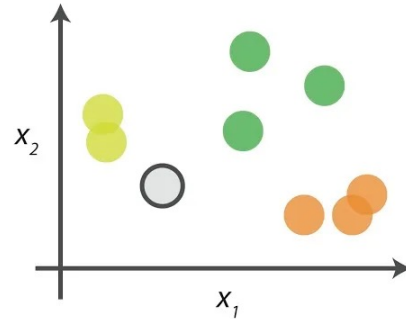


Figure 2.24 K-nearest neighbor techniques look at the k-nearest point to make a prediction.

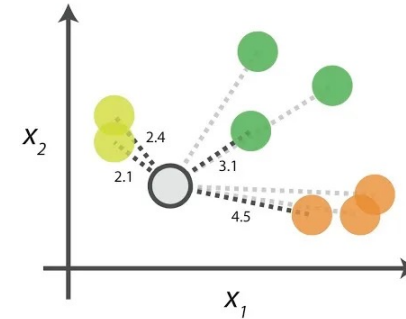
kNN Algorithm

0. Look at the data



Say you want to classify the grey point into a class. Here, there are three potential classes - lime green, green and orange.

1. Calculate distances



Start by calculating the distances between the grey point and all other points.

2. Find neighbours

Point	Distance	
 	2.1	→ 1st NN
 	2.4	→ 2nd NN
 	3.1	→ 3rd NN
 	4.5	→ 4th NN

Next, find the nearest neighbours by ranking points by increasing distance. The nearest neighbours (NNs) of the grey point are the ones closest in dataspace.

3. Vote on labels

Class	# of votes	
	2	→ Class  wins the vote! Point  is therefore predicted to be of class  .
	1	
	1	

Vote on the predicted class labels based on the classes of the k nearest neighbours. Here, the labels were predicted based on the $k=3$ nearest neighbours.

วิธี Re-substitution

```
In [1]: from sklearn.datasets import load_iris  
iris = load_iris()  
X = iris.data  
y = iris.target
```

```
In [2]: from sklearn.neighbors import KNeighborsClassifier  
model = KNeighborsClassifier(n_neighbors=1)
```

```
In [3]: model.fit(X, y)  
y_model = model.predict(X)
```

```
In [4]: from sklearn.metrics import accuracy_score  
accuracy_score(y, y_model)
```

```
Out[4]: 1.0
```

วิธี Hold-out

```
In [5]: from sklearn.model_selection import train_test_split

# split the data with 50% in each set
X1, X2, y1, y2 = train_test_split(X, y, random_state=0, train_size=0.5)

# fit the model on one set of data
model.fit(X1, y1)

# evaluate the model on the second set of data
y2_model = model.predict(X2)
accuracy_score(y2, y2_model)

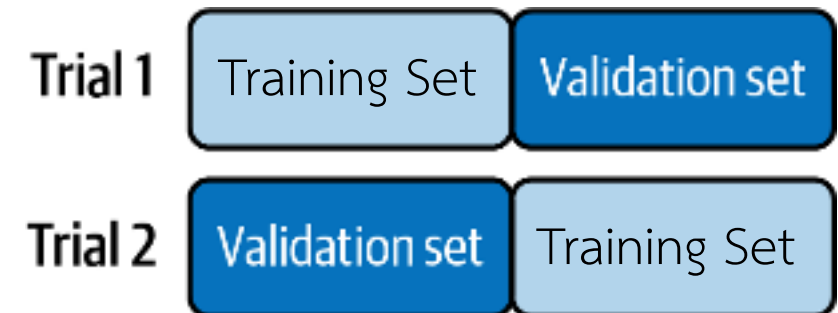
Out[5]: 0.9066666666666666
```

2-folds Cross Validation

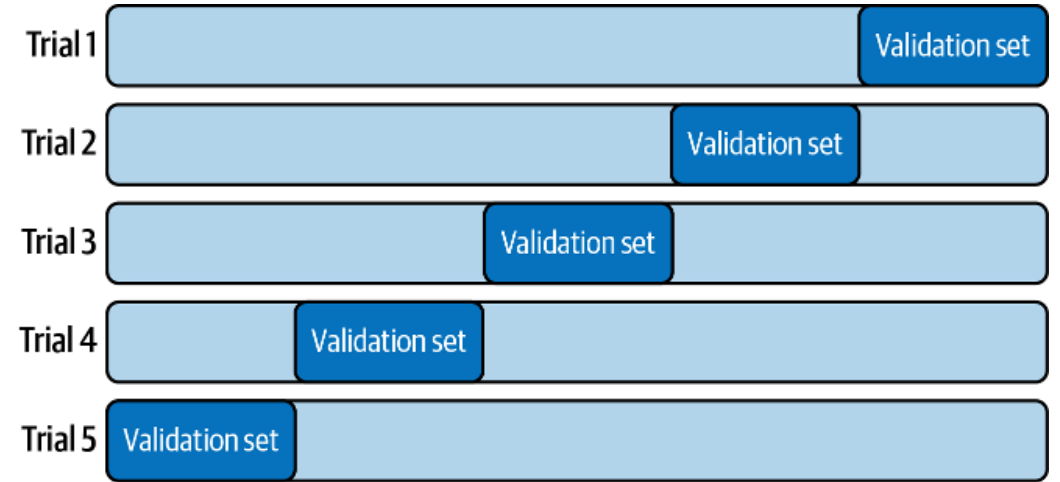
- จากวิธี Hold-out ด้วยการแบ่งข้อมูล 50% ทำให้เกิดการแบ่งข้อมูลออกเป็น 2 ส่วน
- จะสังเกตว่า ใน Trial 1 จะสามารถใช้ข้อมูลส่วนหนึ่งในการ Train แบบจำลอง และอีกส่วนในการ Validate
- ในขณะที่ Trial 2 สามารถกระทำได้เหมือนกันเพียงสลับชุดข้อมูล Train และ Validate

```
In [6]: y2_model = model.fit(X1, y1).predict(X2)
        y1_model = model.fit(X2, y2).predict(X1)
        accuracy_score(y1, y1_model),
        accuracy_score(y2, y2_model)
```

```
Out[6]: (0.96, 0.9066666666666666)
```



5-folds Cross Validation



```
In [7]: from sklearn.model_selection import cross_val_score  
cross_val_score(model, X, y, cv=5)
```

```
Out[7]: array([0.96666667, 0.96666667, 0.93333333, 0.93333333, 1.  ])
```

วิธี leave-one-out cross-validation

- ทำงานเหมือน k-folds
- สุ่ม 1 ตัวอย่างไว้เป็น validation
- ใช้เวลานานในการฝึกสอนแบบจำลองแต่เหมาะกับข้อมูลที่มีจำนวนตัวอย่างน้อย

```
In [8]: from sklearn.model_selection import LeaveOneOut  
        scores = cross_val_score(model, X, y, cv=LeaveOneOut())
```

```
In [9]: scores.mean()
```

```
Out[9]: 0.96
```

แบบฝึกหัดที่ #1

- ศึกษา Data Exploration ข้อมูลดอก Iris:
<http://www.cse.msu.edu/~ptan/dmbook/tutorials/tutorial3/tutorial3.ipynb>
- เปรียบเทียบประสิทธิภาพแบบจำลอง Naïve Bayes และ kNN ด้วยวิธี 10-folds cross-validation
- ทำการทดลองซ้ำ 30 ครั้ง โดยเปลี่ยน Seed number บันทึกผล

```
from sklearn.model_selection import Kfold  
  
cv = KFold(n_splits=10, random_state=1, shuffle=True)  
  
cross_val_score(model, X, y, cv=cv)
```

แบบฝึกหัดที่ #2

- ใช้ข้อมูล California Housing Dataset
- ทำความเข้าใจข้อมูล
- เตรียมข้อมูลด้วย dataframe
- ตรวจสอบ Correlation และ boxplot
- ทดลอง 10-folds cross-validation
- สร้างแบบจำลองทำนายค่า target

```
import pandas as pd
from sklearn.datasets import fetch_california_housing

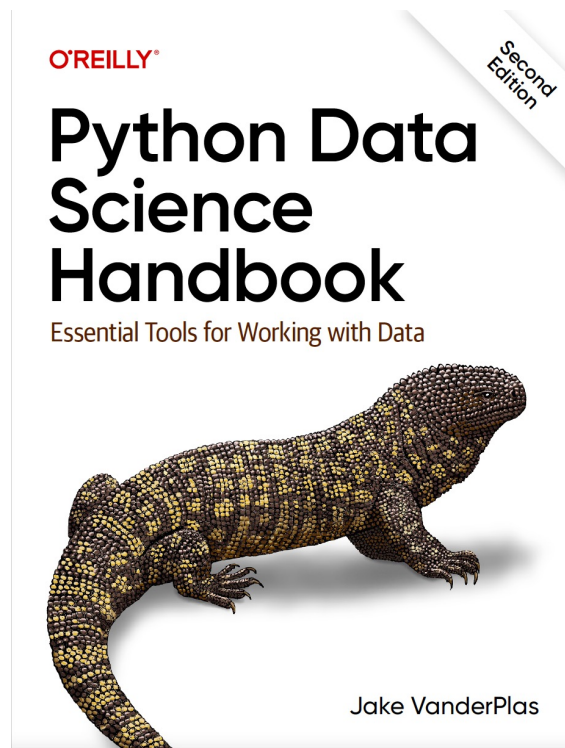
# Load the California Housing dataset
cal_housing = fetch_california_housing()
df = pd.DataFrame(cal_housing.data,
                  columns=cal_housing.feature_names)

# Add the target variable to the DataFrame
df['target'] = cal_housing.target

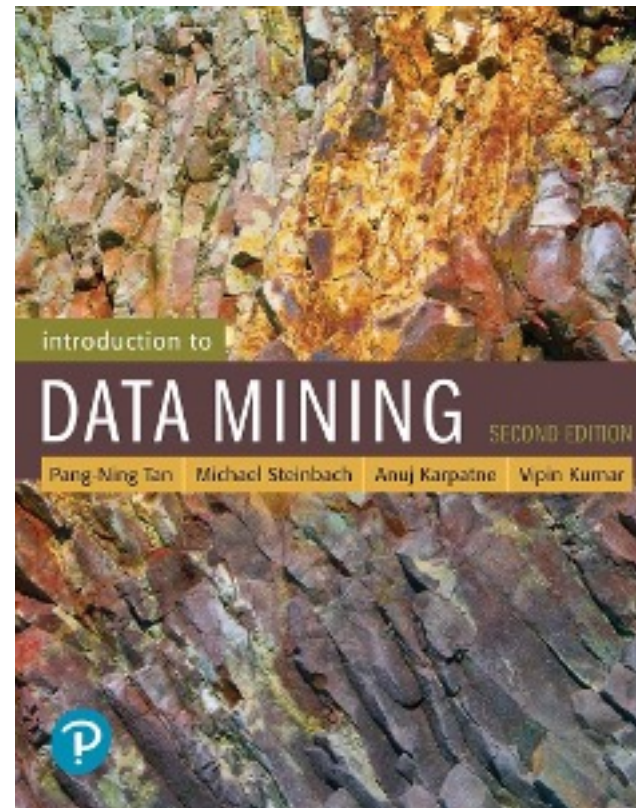
df.sample(n=4000) # Try 400, 4000, 10000, all
```

```
cross_val_score(regressor, X, y, cv=cv, scoring='r2')
```

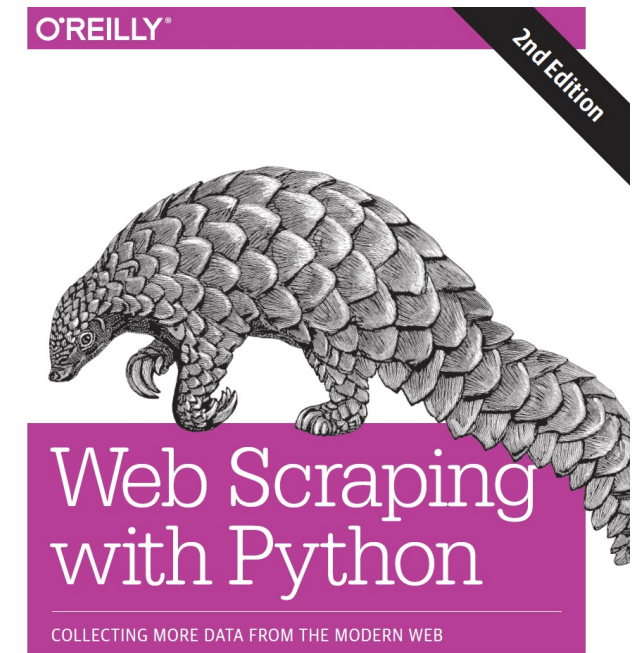
Ref



<https://github.com/jakevdp/PythonDataScienceHandbook>



<https://www-users.cse.umn.edu/~kumar001/dmbook/index.php>



Ryan Mitchell

Q & A